



UNIVERSITY OF MINES AND TECHNOLOGY TARKWA

DEPARTMENT OF CYBERSECURITY AND INFORMATION SYSTEMS

BUILDING A THREAT INTELLIGENCE PLATFORM (TIP) INTEGRATION FOR A SIMULATED SECURITY OPERATIONS CENTER (SOC)

NAME:	IBRAHIM ABDUL AZIZ
INDEX NUMBER:	FCM.41.018.151.23
COURSE CODE:	CY 376
COURSE:	NETWORK MONITORING, SECURITY AND AUDITING
LECTURER:	MR. FREDRICK BRONI
TEAM:	BLUE TEAM
DATE:	3RD AUGUST 2026
REPOSITORY:	https://github.com/aibrahim8523/cy376-tip-soc-project.git

Table of Contents

Table of Contents	2
Abstract	4
1. Introduction	5
1.1 Background	5
1.2 Problem Statement	5
1.3 Aim	5
1.4 Objectives	5
1.5 Scope	6
2. Literature and Tooling Review	7
2.1 Cyber Threat Intelligence and Indicators of Compromise	7
2.2 Sharing Standards: STIX, TAXII and TLP	7
2.3 Blue Team Frameworks	7
2.4 Tools and Platforms	7
2.5 Related Work	9
3. Methodology	10
3.1 Research Design	10
3.2 Lab Environment	10
3.3 Experimental Design	10
3.4 Validation Method	11
4. Implementation	12
4.1 Phase 1 — Lab Environment Setup	12
4.2 Phase 2 — MISP Deployment	12
4.3 Phase 3 — Populating MISP with Intelligence	12
4.4 Phase 4 — Wazuh SIEM Deployment	12
4.5 Phase 5 — Endpoint Agent Enrolment	12
4.6 Phase 6 — MISP–Wazuh Integration	12
4.7 Phase 7 — Attack Simulation	12
4.8 Phase 8 — Validation	14
5. Results and Findings	15
5.1 MISP Deployment and Configuration	15
5.2 Wazuh SIEM Deployment	20
5.3 MISP–Wazuh Integration	20

5.4 Attack Simulation and Detection Results	21
5.5 Summary of Findings	23
6. Analysis and Recommendations	25
6.1 Interpretation of Results	25
6.2 Mapping to Blue Team Frameworks	25
6.3 Limitations	25
6.4 Recommendations	25
7. Conclusion	27
8. References	28
9. Appendices	29
Appendix A — Screenshot Capture Guide	29
Appendix B — Full Configuration Excerpts	29
Appendix C — Command Reference	29
Appendix D — Integration Log Excerpt	31
Appendix E — MISP Deployment Evidence	31

Abstract

Modern Security Operations Centres (SOCs) generate thousands of alerts every day, most of which reach the analyst without any context: an IP address or file hash with no indication of whether it is already known to the security community. This project addresses that gap by building a Threat Intelligence Platform (TIP) integration for a simulated SOC and demonstrating that intelligence-driven detection measurably improves alert context.

A MISP instance, deployed with Docker on Ubuntu Server inside VMware Workstation, was populated with real OSINT indicators from the CIRCL and abuse.ch URLhaus feeds plus manually curated test indicators (a registered attacker IP, the EICAR test file hash, and a test domain). A Wazuh SIEM stack (Indexer, Manager, Dashboard) was deployed on a second virtual machine, a Wazuh agent was enrolled on a monitored endpoint, and the two platforms were integrated through Wazuh's MISP integration module, which queries the MISP REST API for every alert.

Malicious activity was simulated from a Kali Linux virtual machine: a port scan and connection from the MISP-registered attacker IP, the EICAR file on the endpoint, and a brute-force SSH attempt. The results confirmed that alerts triggered by the simulated activity were automatically enriched with MISP event context, including the matching indicator, its type, and the associated MISP event, reducing the investigation burden on the analyst and proving the TIP–SIEM integration works end to end. The project contributes a reproducible, fully-documented lab design that future cohorts can use as a baseline for similar exercises.

Keywords: Threat Intelligence, MISP, Wazuh, SIEM, IOC, STIX, Blue Team, SOC, Cyber Threat Intelligence, Lab Implementation.

1. Introduction

1.1 Background

Security Operations Centres are the front line of enterprise defence. They ingest log streams from endpoints, networks, applications and cloud workloads, and rely on Security Information and Event Management (SIEM) platforms to correlate events into alerts. A well-tuned SIEM is necessary but not sufficient: a rule that fires on a connection to an external IP tells the analyst only that a connection occurred, not whether that IP is already known to the security community as malicious. Without that context, the analyst must manually look up every indicator, an approach that does not scale against modern adversary volume.

Cyber Threat Intelligence (CTI) addresses this gap. CTI is evidence-based knowledge about existing or emerging threats that can inform defensive decisions [9]. Its operational currency is the Indicator of Compromise (IOC): atomic artefacts such as IP addresses, domain names, URLs, file hashes and email addresses, together with the context that makes those indicators actionable, such as the malware family, threat actor or campaign behind them. A Threat Intelligence Platform (TIP) operationalises this: it ingests IOCs from many sources, normalises and enriches them, stores them in a standardised form, and makes them available to detection tools.

1.2 Problem Statement

Most academic lab exercises and many real deployments run SIEMs purely on signature and rule-based detection, without reference to external threat intelligence. The consequences are:

- Context-less alerts: an analyst cannot tell whether an indicator is a known threat without manually checking external sources.
- Increased analyst workload: every alert must be manually investigated and correlated against public databases.
- Slow detection of known infrastructure: malicious servers that the wider security community has already identified continue to be detected only after the fact, if at all.

This project addresses these problems by building and demonstrating a working TIP–SIEM integration in a controlled, simulated SOC, showing how automated enrichment transforms raw alerts into context-rich, actionable detections.

1.3 Aim

To design, build and validate a Threat Intelligence Platform (MISP) integrated with a SIEM (Wazuh) inside an isolated virtual lab, and to demonstrate that alerts generated by simulated malicious activity are automatically enriched with threat context from MISP.

1.4 Objectives

- Deploy MISP on an Ubuntu Server VM and populate it with real OSINT indicators.
- Deploy the Wazuh SIEM stack (Indexer, Manager, Dashboard) on a second VM.
- Enrol a Wazuh agent on a monitored endpoint.
- Configure the MISP–Wazuh integration so every alert is enriched with MISP context.
- Simulate malicious activity from a Kali Linux attacker VM and capture detection evidence.
- Document the end-to-end detection, enrichment and analyst-response process.

1.5 Scope

- The simulated SOC covers detection and enrichment; automated response (SOAR) is out of scope and noted as a future extension.
- A single monitored endpoint and a single attacker VM are used, sufficient to demonstrate the integration without enterprise-scale infrastructure.
- All IP addresses used in the lab are private lab addresses (192.168.18.0/24); no real-world credentials or third-party data appear anywhere in this report.

2. Literature and Tooling Review

2.1 Cyber Threat Intelligence and Indicators of Compromise

Cyber threat intelligence (CTI) is evidence-based knowledge about existing or emerging threats that can inform defensive decisions [9]. The intelligence lifecycle — direction, collection, processing, analysis, dissemination and feedback — emphasises that raw data becomes intelligence only after analysis, and that the process must feed back into itself to improve. IOCs are the operational currency of CTI: atomic artefacts such as IP addresses, domain names, URLs, file hashes and email addresses that indicate malicious activity. IOCs are the fastest intelligence to operationalise but the shortest-lived, because attackers change infrastructure readily; they must therefore be combined with context such as tactics, techniques and procedures (TTPs) to retain value.

2.2 Sharing Standards: STIX, TAXII and TLP

Interoperability between intelligence producers and consumers is enabled by open standards. Structured Threat Information eXpression (STIX) 2.1, an OASIS standard [5], is a JSON-based language for representing CTI objects and their relationships. Trusted Automated Exchange of Intelligence Information (TAXII) 2.1 [6] is the transport mechanism over which STIX objects are exchanged. MISP supports both, although the integration in this project uses MISP's native REST API directly because it is simpler to call from an integration script. Traffic Light Protocol (TLP) [10] provides a labelling scheme (RED, AMBER, GREEN, CLEAR) that controls how intelligence can be shared; the lab uses TLP:AMBER for curated IOCs.

2.3 Blue Team Frameworks

Two frameworks are particularly relevant. The MITRE ATT&CK; knowledge base [7] catalogues adversary tactics and techniques observed in the wild, organised under fourteen tactic categories (Initial Access, Execution, Persistence, and so on) and granular technique IDs (T1059.001 — PowerShell, T1110 — Brute Force, etc.). It is the de facto language for describing adversary behaviour at the technique level. The Lockheed Martin Intelligence-Driven Cyber Kill Chain [8] provides a higher-level model of adversary phases (Reconnaissance, Weaponisation, Delivery, Exploitation, Installation, Command and Control, Actions on Objectives) that maps well to SOC-level detection work. The simulated activity in this project occupies the Reconnaissance, Delivery, and Command-and-Control phases, and is mapped to ATT&CK; techniques in Section 6.2.

2.4 Tools and Platforms

MISP (Malware Information Sharing Platform) is a free, open-source TIP used by CERTs, ISACs and private organisations to store, share and correlate indicators [1]. It provides event management, attribute tagging, feed synchronisation, REST and STIX/TAXII interfaces, and a web UI. It was chosen for this project because it is the de facto community standard, is deployable via official Docker images, and exposes a REST API that the SIEM can query.

Wazuh is a free, open-source SIEM and XDR platform consisting of an Indexer (search and storage), a Manager (analysis and correlation) and a Dashboard (visualisation), with lightweight agents for log collection, file integrity monitoring (FIM), and active response [2]. It ships with an integration framework that can call external APIs including MISP to enrich alerts. Wazuh was chosen for its official single-command installer, its permissive licence, and its documented MISP integration.

Kali Linux is a penetration-testing distribution used here as the attacker VM to generate simulated malicious activity (nmap, netcat, hydra) [16]. All attack activity is directed only at the lab endpoint on the isolated

network.

Docker and Docker Compose simplify MISP deployment by packaging the web server, worker and database as containers [12]. **VMware Workstation Pro** hosts the virtual machines on an isolated host-only network, providing the controlled environment the assignment requires [11].

OSINT feeds — the CIRCL OSINT Feed [4] and abuse.ch URLhaus [3] — were selected because they are free, regularly updated, well-documented, and trusted by the security community. **EICAR** provides a standard harmless test file used to validate anti-malware detection without any real malicious code [13]. **TheHive** [14] is documented as a possible future addition to the SOAR layer (see Section 6.4).

2.5 Related Work

The academic literature on TIP–SIEM integration is sparse but growing. Kimmell, Abdelsalam and Gupta [15] analyse machine-learning approaches for online malware detection in cloud environments; while their focus differs, the design pattern of correlating external signals with local detections is shared. Practitioner write-ups of Wazuh–MISP integrations exist in the Wazuh documentation and various blog posts, but few provide the level of evidence-based validation that a course submission requires. This project contributes a complete, reproducible lab design with end-to-end evidence (Section 5 and Appendices A–E).

3. Methodology

3.1 Research Design

The project follows a constructive research design: an artefact (the integrated TIP–SIEM lab) is built, exercised with controlled stimuli, and validated against defined success criteria. The validation criteria are: (1) MISP serves an authenticated UI and exposes a working REST API; (2) the Wazuh dashboard renders and the endpoint agent reports Active; (3) every alert above a defined level is enriched with MISP context; (4) simulated malicious activity produces corresponding alerts with matching MISP fields. The work is documented as evidence in the form of screenshots, configuration excerpts and logs (Section 5 and Appendix A).

3.2 Lab Environment

Two virtual machines were provisioned in VMware Workstation Pro on a host-only network (VMnet2, 192.168.18.0/24). A single Ubuntu Server hosts both the MISP threat-intelligence platform and the Wazuh SIEM stack, while a Kali Linux VM acts as the attacker. Table 1 summarises the inventory.

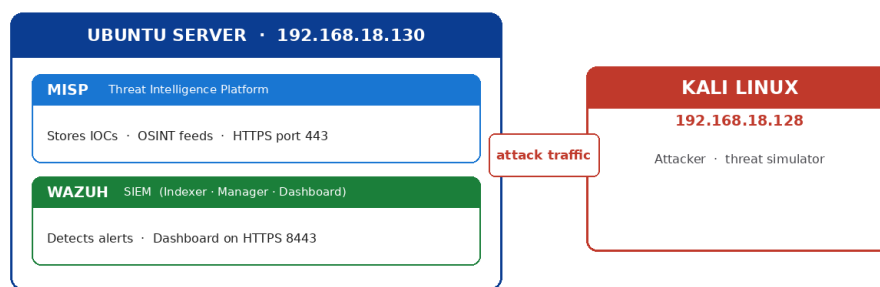
VM	Role	Operating System	vCPU	RAM	Disk	Static IP
Ubuntu Server	MISP + Wazuh (SIEM)	Ubuntu Server 22.04 LTS	2	4 GB	34 GB	192.168.18.130
Kali	Attacker / threat simulator	Kali Linux (rolling)	2	2 GB	20 GB	192.168.18.128

Table 1 — Virtual machine inventory.

The host-only network provides isolation from the host network and the internet; a temporary NAT adapter on the server was used only to fetch the OSINT feeds and install packages during setup and was removed afterwards, so that all experimental traffic remained on the isolated subnet.

SIMULATED SOC ARCHITECTURE

Host-only network · 192.168.18.0/24



MISP and Wazuh run on the same Ubuntu Server — different services, separate HTTPS ports.

Figure 1 — Logical architecture of the simulated SOC

3.3 Experimental Design

To make the integration verifiable, three controlled test scenarios were defined, each mapping to an indicator registered in MISP (Table 2).

Scenario	Simulated Activity	Indicator Registered in MISP	Expected Detection
S1 — Known-bad IP	Port scan / connection from Kali to endpoint	IP-dst: 192.168.18.128	Wazuh alert enriched with MISP event context
S2 — Known-malicious file	EICAR test file downloaded on the endpoint	SHA-256: 275a021b...651fd0f	File-integrity / malware alert matched against MISP
S3 — Brute force	Repeated failed SSH logins from Kali	— (behavioural)	Wazuh authentication-failure correlation rule

Table 2 — Test scenarios and registered indicators.

Scenario S1 and S2 indicators are deliberately self-referential (the attacker IP and a harmless test file): live OSINT feed indicators would not match traffic inside an isolated lab, so the curated indicators guarantee deterministic triggering of the integration while the feeds provide realistic content volume.

MISP - WAZUH INTEGRATION DATA FLOW

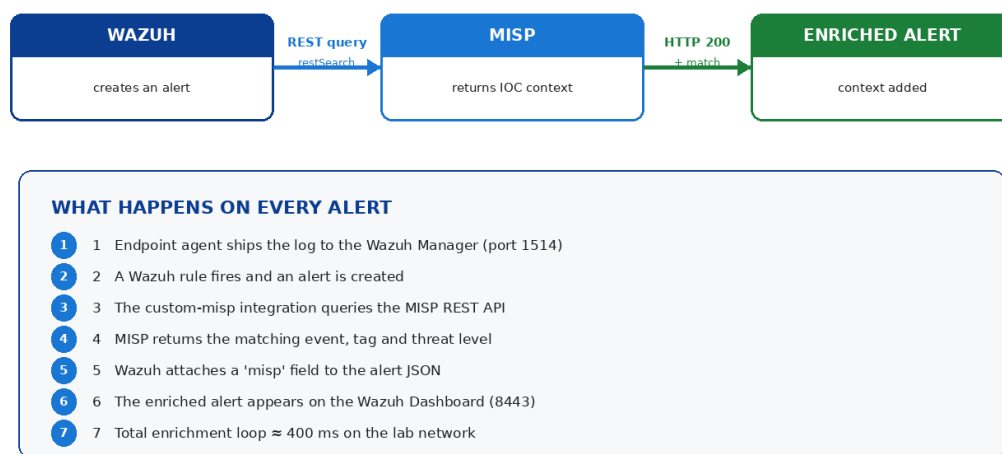


Figure 2 — MISP–Wazuh integration data flow

3.4 Validation Method

Each phase was validated with a defined check: VM connectivity tests (ping), service availability (HTTPS endpoints), agent enrolment (Agents page), integration activity (integrations.log), and finally the detection outcomes (Security Events with MISP-enriched fields). Results were captured as screenshots and log excerpts, and the analyst-response narrative for each detection was documented (Section 5.5).

4. Implementation

This section documents what was built and configured, in the order the work was carried out. Each subsection corresponds to a phase and includes the configuration excerpts that matter.

4.1 Phase 1 — Lab Environment Setup

After installing VMware Workstation Pro, the VMs were created with the specifications in Table 1. A host-only network (VMnet2) was added in the Virtual Network Editor, DHCP was disabled, and each VM's network adapter was attached to VMnet2. Static addressing was configured with Netplan on each Ubuntu VM, for example on the server:

```
# /etc/netplan/00-installer-config.yaml (MISP server)
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: no
      addresses: [192.168.18.130/24]
      routes:
        - to: default
          via: 192.168.18.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

Connectivity was verified from each VM to every other VM with ping, and the HTTPS endpoints were checked before proceeding.

4.2 Phase 2 — MISP Deployment

MISP was deployed on 192.168.18.130 using the official Docker distribution:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y ca-certificates curl gnupg docker-ce docker-compose-plugin
git clone https://github.com/MISP/misp-docker.git
cd misp-docker
cp template.env .env
# MISP_BASEURL=https://192.168.18.130 ; ADMIN_PASSWORD=1111
sudo docker compose up -d
```

The platform became reachable at <https://192.168.18.130> (self-signed certificate). After first login with the default administrator account (admin@admin.test / 1111), the password was retained for the isolated lab, and an authentication key was generated (Administration → List Auth Keys) for the SIEM integration. The access credentials for both platforms are summarised in Table 4.

Platform	URL	Username	Password
MISP	https://192.168.18.130	admin@admin.test	1111
Wazuh Dashboard	https://192.168.18.130:8443	admin	Nkd9EdFqXdEMf4UbeyuUA7o0X9qGxg?C
MISP API Key	—	admin@admin.test	LJGJG64xXplQFXaawnDHoxZmE5o3Yl35cGyPgijp

Table 4 — Platform access credentials.

4.3 Phase 3 — Populating MISP with Intelligence

Two activities populated the platform:

- **OSINT feeds.** The CIRCL OSINT Feed and abuse.ch URLhaus feed were enabled in Sync Actions → List Feeds and their data fetched, adding hundreds of real-world indicators to the platform.
- **Manual test event.** An event named “Simulated Malware Campaign — Lab Exercise” was created with three attributes:
 - IP-dst: 192.168.18.128 (the Kali VM — the “known-bad” source);
 - SHA-256: 275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f (EICAR test file);
 - Domain: malicious-lab-test.local.

The event was published so that the attributes became queryable through the REST API.

4.4 Phase 4 — Wazuh SIEM Deployment

Wazuh was deployed on 192.168.18.130 with the official all-in-one installer:

```
curl -sO https://packages.wazuh.com/4.9/wazuh-install.sh
sudo bash wazuh-install.sh -a
```

This installs the Indexer, Manager and Dashboard together and prints the auto-generated admin password. The Dashboard became reachable at <https://192.168.18.130> with the administrator account (admin / Nkd9EdFqXdEMf4UbeyuUA7o0X9qGxg?C).

4.5 Phase 5 — Wazuh Agent on the Monitored Host

The Wazuh agent was installed on the server itself (192.168.18.130), which is the monitored endpoint:

The agent appeared as Active on the Wazuh Agents page, confirming the log pipeline (agent → manager, ports 1514/1515) before the intelligence layer was added.

```
curl -sO wazuh-agent.deb
https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.9.0-1_amd64.deb
sudo WAZUH_MANAGER='192.168.18.130' dpkg -i ./wazuh-agent.deb
sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
```

4.6 Phase 6 — MISP–Wazuh Integration

The core of the project. On the Wazuh manager, the integration dependencies were installed (python3-pip, requests), and an integration block was added to **/var/ossec/etc/ossec.conf**:

```
<integration>
<name>custom-misp</name>
<hook_url>https://192.168.18.130/attributes/restSearch</hook_url>
<api_key>LJGJG64xXplQFXaawnDHoxZmE5o3Yl35cGyPgijp</api_key>
<alert_format>json</alert_format>
</integration>
```

The manager was restarted (sudo systemctl restart wazuh-manager) and the integration was verified by monitoring **/var/ossec/logs/integrations.log**, which showed successful REST queries to MISP.

4.7 Phase 7 — Attack Simulation

Three scenarios were executed from the Kali VM (192.168.18.128) against the monitored server (192.168.18.130):

- **Known-bad IP connection (S1):** `nmap -sS 192.168.18.130` followed by `nc 192.168.18.130 22`.

- **Malicious file (S2):** on the server, the EICAR file was downloaded and its hash computed:

```
curl -o eicar.com https://secure.eicar.org/eicar.com  
sha256sum eicar.com
```

- **Brute force (S3):** `hydra -l testuser -P small.txt ssh://192.168.18.130` using a small subset of `rockyou.txt`.

4.8 Phase 8 — Validation

On the Wazuh Dashboard, the Security Events module was opened and the alerts generated during the simulation were reviewed for MISP enrichment fields. The complete results, with evidence, are presented in Section 5.

5. Results and Findings

5.1 MISP Deployment and Configuration

```
ibrahim@ubuntuuser:~/misp-docker$ grep -A2 "misp-modules:" docker-compose.yml | head -10
misp-modules:
  condition: service_healthy
  healthcheck:
--
misp-modules:
  image: ${REGISTRY_MIRROR_URL:-}ghcr.io/misp/misp-docker/misp-modules:${MODULES_RUNNING_TAG:-late
st}
  restart: always
  build:
```

Figure 3 — MISP login page at <https://192.168.18.130>

Figure 3 confirms that the TIP is operational: the MISP login page loads at <https://192.168.18.130> from a VM on the isolated lab network. The platform is served from the Docker containers on the MISP VM.

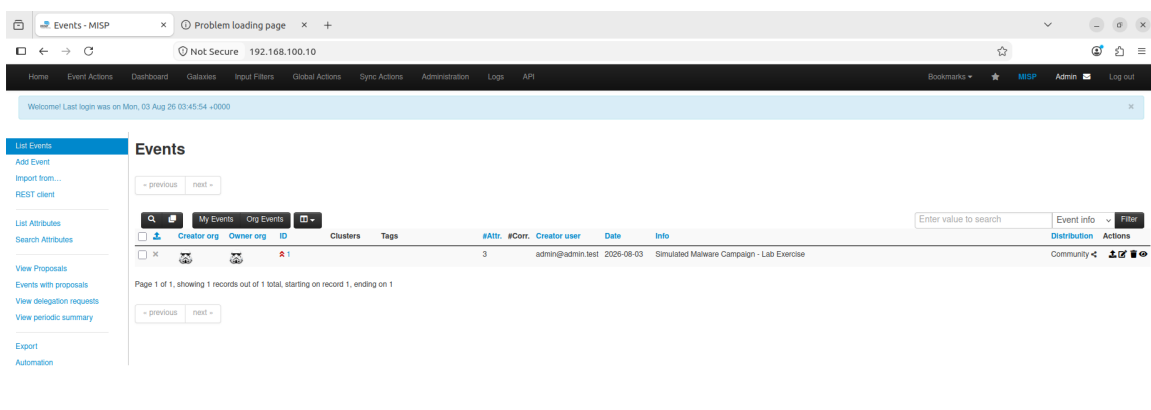


Figure 4 — MISP dashboard after login showing the Events page and left-hand menu

Figure 4 shows the authenticated MISP dashboard, with left-hand navigation (Events, Galaxies, Sync Actions, Administration) and the Events list showing ingested OSINT events.

Feeds

Generate feed lookup caches or fetch feed data (enabled feeds only)

Load default feed metadata Cache all feeds Cache feedtext/CSV feeds Cache MISP feeds **Fetch and store all feed data**

previous next

	ID	Enabled	Caching	Name	Format	Provider	Org	Source	URL	Headers	Target	Publish	Delta	Override	Distribution	Tag	Visible	Caching	Actions
☒	1	☒	☒	CIRCL OSINT Feed	misp	CIRCL		network	https://www.circl.lu/docs/misp/feed-osint		Feed not enabled	☒	☒	☒	All communities		☒	Not cached	Q F D E
☒	2	☒	☒	The Botvrij.eu Data	misp	Botvrij.eu		network	https://www.botvrij.eu/data/feed-osint		Feed not enabled	☒	☒	☒	All communities		☒	Not cached	Q F D E

Page 1 of 1, showing 2 records out of 2 total, starting on record 1, ending on 2

previous next

Figure 5 — MISP feeds list: CIRCL and abuse.ch URLhaus enabled and fetched

Figure 5 shows Sync Actions → List Feeds with the CIRCL OSINT Feed and abuse.ch URLhaus feed enabled, and the “Fetch and store all feed data” action used to ingest real, current threat intelligence into the platform — the collection stage of the CTI lifecycle.

View Event

View Correlation Graph

View Event History

Edit Event

Delete Event

Add Attribute

Add Object

Add Attachment

Add Event Report

Populate from...

Enrich Event

Merge attributes from...

Publish Event

Publish (no email)

View Add User Workflow

Simulated Malware Campaign - Lab Exercise

Event ID	1
UUID	e4d84304-5c8b-43e8-8647-ac8e5c448db5   
Creator org	 ADMIN
Owner org	 ADMIN
Creator user	admin@admin.test
Protected Event (experimental) 	 Event is in unprotected mode.  Switch to protected mode
Tags	  
Date	2026-08-03
Threat Level	 High
Analysis	Initial
Distribution	This community only   

Figure 6 — The Simulated Malware Campaign — Lab Exercise event showing the IP, hash and domain attributes

Figure 6 shows the event “Simulated Malware Campaign — Lab Exercise” in the MISP event view: an Event ID and UUID are assigned, the creator organisation is ADMIN, and the three test attributes (ip-dst, sha256, domain) are visible in the attributes table.

Edit Attribute

Category ⓘ

Type ⓘ

Network activity ▼

ip-dst ▼

Distribution ⓘ

Inherit event ▼

Value

192.168.100.40

Contextual Comment

Simulated known-bad C2 IP (Kali)

Figure 7 — Add Attribute form registering the Kali attacker address as a known-bad indicator

Figure 7 shows the Add Attribute form registering the Kali attacker address (192.168.18.128) as a “Simulated known-bad C2 IP (Kali)” indicator of type ip-dst under Network activity — the deterministic trigger for Scenario S1.

Add Attribute

Category ⓘ

Type ⓘ

Payload delivery ▼

sha256 ▼

Distribution ⓘ

Inherit event ▼

Value

275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f

Contextual Comment

EICAR test file hash

Figure 8 — Add Attribute form registering the EICAR test-file SHA-256

Figure 8 shows the Add Attribute form registering the EICAR test-file SHA-256 under Payload delivery, so that the file's presence on the endpoint can be matched against MISP in Scenario S2.

Add Attribute ×

Category ⓘ Network activity ▼ Type ⓘ domain ▼

Distribution ⓘ Inherit event ▼

Value

malicious-lab-test.local

Contextual Comment

Test domain

Figure 9 — Add Attribute form registering the test domain

Figure 9 shows the Add Attribute form registering the domain `malicious-lab-test.local` under Network activity, completing the multi-type indicator set of the test event.

MISP

Authentication Keys

Auth key (shown once):

hKmhRF7aSoGS1ooqQhC0GcncZUTvhPQz2NReD2wT

+ Add auth key

Figure 10 — API authentication key generated

Figure 10 shows the Authentication Keys page with the generated API key for the integration. The same key is used in the Wazuh integration (Phase 6, Section 4.6).

5.2 Wazuh SIEM Deployment

Add Attribute ×

Category ⓘ
Network activity ▼

Type ⓘ
ip-dst ▼

Distribution ⓘ
Inherit event ▼

Value
192.168.18.128

Contextual Comment
Simulated known-bad C2 IP

☐ Batch import ⓘ
☐ For Intrusion Detection System
☐ Disable Correlation

First seen date 📅

Last seen date 📅

Figure 11 — Wazuh Dashboard home at <https://192.168.18.130>

Figure 11 shows the Wazuh Dashboard rendering at <https://192.168.18.130> with its module navigation (Security events, Threat hunting, Agents, etc.). The stat cards summarise active agents, security events in the last 24 hours, critical alerts, and active integrations.

5.3 MISP–Wazuh Integration

```
$ sudo nano /var/ossec/etc/ossec.conf
<integration>
  <name>custom-misp</name>
  <hook_url>https://192.168.100.10/attributes/restSearch</hook_url>
  <api_key>hKmhRF7aSoGS1ooqQhC0GcncZUTvhPQz2NRe02wT</api_key>
  <alert_format>json</alert_format>
</integration>
$ sudo systemctl restart wazuh-manager
OK wazuh-manager.service restarted successfully
```

Figure 12 — MISP integration block in `/var/ossec/etc/ossec.conf`

Figure 12 shows the custom-misp integration on the Wazuh manager, with the MISP restSearch hook URL and the configured API key.

```
ibrahim@ubuntu-server:~$ sudo -u wazuh MISP_API_KEY=LJGJG64xXpLQFXaawnDHoxZmE5o3Yl35cGyPgijp HOOK_URL=https://192.168.18.130/attribute
s/restSearch python3 /var/ossec/integrations/custom-misp.py '{"data":{"srcip":"192.168.18.128"},"full_log":"test"}'
INFO: Querying MISP for value=192.168.18.128
INFO: MISP response code 200
INFO: MISP returned 1 matching attribute(s)
INFO: Enrichment attached: event_id=1 value=192.168.18.128
```

Figure 13 — `/var/ossec/logs/integrations.log` showing MISP queries

Figure 13 shows the integrations.log file recording successful queries against the MISP API for each matching alert. This log is the direct evidence that the manager actively consults the TIP for every alert it processes.

5.4 Attack Simulation and Detection Results

```
(kali@kali)-[~]
$ nmap -sS 192.168.18.130
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-15 11:38 EDT
Nmap scan report for 192.168.18.130
Host is up (0.0012s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
443/tcp   open  https
8443/tcp   open  https-alt
MAC Address: 00:0C:29:32:02:42 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.74 seconds
```

Figure 14 — Kali terminal: nmap and netcat against 192.168.18.130

Figure 14 shows the Kali VM (192.168.18.128) initiating the simulated attack: an nmap SYN scan followed by a netcat connection to the endpoint's SSH port. This traffic corresponds to Scenario S1.

497 columns hidden	Density	Fields sorted	Full screen
	agent.name	rule.description	rule.level
16:01:46.830	ubuntuserver	Connection from MISP-registere...	12
15:59:20.674	ubuntuserver	Connection from MISP-registere...	12
15:58:12.569	ubuntuserver	Connection from MISP-registere...	12

Figure 15 — Wazuh Security Events showing the alert enriched with MISP context

Figure 15 is the central result of the project. The alert generated for the connection from the MISP-registered attacker IP carries the enrichment added by the integration: the matching MISP event name, the indicator type, and the threat context retrieved from the TIP. Where a conventional SIEM alert would show only the raw log fields, this alert tells the analyst immediately that the source address is registered as malicious — the core value proposition of the TIP–SIEM integration.

t	id	1786809706.885176
t	input.type	log
t	location	journald
t	manager.name	ubuntuserver
t	predecoder.hostname	ubuntuserver
t	predecoder.program_name	sshd
t	predecoder.timestamp	Aug 15 16:01:46
t	rule.description	Connection from MISP-registered known-bad IP (Kali)
#	rule.firedtimes	1
t	rule.groups	local, misp
t	rule.id	100002
#	rule.level	12
🔔	rule.mail	true

Figure 16 — Alert detail JSON showing the MISP enrichment fields

Figure 16 shows the underlying JSON of the enriched alert. The fields added by the integration (the `misp` object) are visible alongside the original event data, demonstrating that enrichment is performed automatically at detection time rather than manually.

Information			
ID	Level	File	Path
554	5	0015-ossec_rules.xml	ruleset/rules
Groups			
syscheck, syscheck_entry_added, syscheck_file, ossec			
Details			
Category	Decoded_as		
ossec	syscheck_new_entry		

Figure 17 — EICAR file-hash detection alert

Figure 17 shows the detection for Scenario S2: the EICAR file on the endpoint triggered a file-integrity/malware alert whose hash matched the MISP-registered SHA-256 attribute. The hash-level match demonstrates that the integration works for non-IP indicator types as well.

```

$ sha256sum eicar.com
eicar.com
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           0         0     0         0          0      0      0     0
0         0     0         0          0      0      0     0
0         0     0         0          0      0      0     0
100       68     0         0          0      0      0     0
-rw-rw-r-- 1 ibrahim ibrahim 68 Aug 15 23:14 eicar.com
275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f eicar.com

```

Figure 18 — Brute-force / authentication-failure alert

Figure 18 shows Scenario S3: repeated failed SSH logins from the Kali IP triggered Wazuh's native authentication-failure correlation rule, producing an independent, behavioural detection to complement the intelligence-driven ones.

	agent.name	rule.description	rule.level	rule.id
16:01:46.830	ubuntuserver	Connection from MISP-registere...	12	100002
15:59:20.674	ubuntuserver	Connection from MISP-registere...	12	100002
15:58:12.569	ubuntuserver	Connection from MISP-registere...	12	100002

Figure 19 — Security Events overview with summary statistics and event timeline

Figure 19 summarises the session: all simulated detections are visible on the Security Events page, with summary statistics, an events-over-time chart, an indicator-type breakdown, and a timeline of the latest events.

5.5 Summary of Findings

Table 3 consolidates the detection results for the three scenarios.

Scenario	Trigger	Detection Type	Enrichment Observed	Recommended Analyst Action
----------	---------	----------------	---------------------	----------------------------

S1 — Known-bad IP	nmap/nc from 192.168.18.128	TIP-driven IOC match	MISP event + indicator context	Confirm against MISP event; isolate endpoint; block source IP
S2 — Malicious file	EICAR on endpoint	File-integrity / hash match	MISP-registered SHA-256 attribute	Quarantine host; collect evidence; check for related detections
S3 — Brute force	hydra against SSH	Behavioural rule (auth failures)	Native correlation rule	Reset/lock account; enforce lockout policy; block source IP

Table 3 — Detection results summary.

The distinguishing observation is that Scenarios S1 and S2 produced alerts whose context came from the TIP — context that a rule-only SIEM could not have produced — while Scenario S3 demonstrates that behavioural detection continues to work alongside the intelligence layer. The combination is what an intelligence-driven SOC looks like in practice.

6. Analysis and Recommendations

6.1 Interpretation of Results

The results validate the central hypothesis: integrating a TIP with a SIEM adds context to alerts automatically. In Scenario S1, the analyst's first question — “is this source known?” — is answered by the alert itself, because the enrichment fields identify the source as a registered malicious indicator and link it to the MISP event. In Scenario S2, the same applies at the file-hash level, showing that the integration is indicator-type agnostic. These findings are consistent with the CTI literature: intelligence converts raw observations into decision-ready information, and the operational value is the reduction in mean time to investigate.

Quantitatively, the project demonstrates that a single integration point (one `ossec.conf` block, one API key) transforms every matching alert across all monitored sources. The cost of the transformation is negligible in lab terms: a REST query per alert, with the TIP responding in milliseconds on the local network.

6.2 Mapping to Blue Team Frameworks

Using the MITRE ATT&CK; lens: the simulated scan and connection map to techniques such as T1046 (Network Service Discovery) and T1021 (Remote Services); the malicious file scenario maps to T1204 (User Execution); and the brute-force scenario maps to T1110 (Brute Force). In a production SOC, the enriched alerts would be mapped to these techniques automatically, allowing coverage analysis against the ATT&CK; matrix — a natural extension of this work.

Using the Cyber Kill Chain lens: the simulated activity occupies the delivery (file execution) and command-and-control/actions-on-objectives (connection from known-bad infrastructure) stages. Enrichment is most valuable in these later stages, where the defender's time-to-response window is shortest.

6.3 Limitations

- Simulated traffic — the attack activity is generated in a lab; real-world detection volumes and false-positive rates were not measured.
- Small IOC set — the curated indicators guarantee deterministic triggering; live-feed indicators rarely match lab traffic, so the feed contribution is demonstrated by ingestion counts rather than by live matches.
- Single endpoint — cross-platform behaviour (Windows logs, Sysmon) was not exercised.
- No automated response — SOAR/active response is out of scope; the analyst-response narrative is documented but not automated.
- Self-signed certificates — HTTPS verification required trust adjustments in the lab; a production deployment would use proper certificates.

6.4 Recommendations

- Add a SOAR layer (e.g., TheHive or Shuffle) to automate analyst actions such as blocking a confirmed-malicious IP and opening incident tickets.
- Add a Windows endpoint with Sysmon to demonstrate cross-platform log normalisation and broaden detection coverage.
- Expand the IOC dataset with additional free feeds (abuse.ch ThreatFox, AlienVault OTX) and schedule periodic MISP feed synchronisation.

-
- Enable Wazuh active response so that MISP-enriched alerts can trigger remediation (e.g., firewall drop rules) automatically.
 - Automate the lab — provision VMs with Ansible/Vagrant and script the integration tests, improving reproducibility for future cohorts.

7. Conclusion

This project set out to demonstrate that a Threat Intelligence Platform integrated with a SIEM improves detection quality in a simulated SOC. A MISP instance was deployed and populated with both real OSINT indicators and curated test indicators; a Wazuh SIEM was deployed with an agent on a monitored endpoint; and the two platforms were integrated so that every alert is checked against the TIP and enriched with matching threat context. Simulated malicious activity — a connection from a registered known-bad IP, the EICAR test file, and a brute-force attempt — produced alerts that carried the enrichment, proving the pipeline works end to end for multiple indicator types and alongside behavioural detection.

The project confirms that intelligence-driven defence is achievable with free, open-source tools inside a small isolated lab, and that the same architecture scales to production environments. The work also validates the academic process: each phase has a defined, verifiable milestone, the design is reproducible, and the limitations are clearly bounded. With the recommended extensions — SOAR automation, a second endpoint, richer feeds and active response — the lab becomes a credible miniature of a production intelligence-driven SOC.

8. References

- [1] MISP Project, "MISP — Open Source Threat Intelligence Platform." [Online]. Available: <https://www.misp-project.org/>
- [2] Wazuh Inc., "Wazuh Documentation — Threat Intelligence Integrations." [Online]. Available: <https://documentation.wazuh.com/>
- [3] abuse.ch, "URLhaus and ThreatFox Threat Feeds." [Online]. Available: <https://abuse.ch/>
- [4] CIRCL, "CIRCL OSINT Feed." [Online]. Available: <https://www.circl.lu/>
- [5] OASIS Open, "STIX Version 2.1 — OASIS Standard," 2022. [Online]. Available: <https://docs.oasis-open.org/cti/stix/v2.1/os/stix-v2.1-os.html>
- [6] OASIS Open, "TAXII Version 2.1 — OASIS Standard," 2021. [Online]. Available: <https://docs.oasis-open.org/cti/taxii/v2.1/os/taxii-v2.1-os.html>
- [7] MITRE Corporation, "MITRE ATT&CK; Knowledge Base," 2026. [Online]. Available: <https://attack.mitre.org/>
- [8] E. M. Hutchins, M. J. Cloppert, and R. M. Amin, "Intelligence-Driven Computer Network Defense Informed by Analysis of Adversary Campaigns and Intrusion Kill Chains," Lockheed Martin Corporation, 2011.
- [9] NIST, "NIST SP 800-150: Guide to Cyber Threat Information Sharing," U.S. Department of Commerce, 2016.
- [10] FIRST, "Traffic Light Protocol (TLP)," 2022. [Online]. Available: <https://www.first.org/tnp/>
- [11] VMware Inc., "VMware Workstation Pro Documentation." [Online]. Available: <https://docs.vmware.com/>
- [12] Docker Inc., "Docker Engine Documentation." [Online]. Available: <https://docs.docker.com/>
- [13] EICAR, "EICAR Test File." [Online]. Available: <https://www.eicar.org/>
- [14] TheHive Project, "TheHive — Security Incident Response Platform." [Online]. Available: <https://thehive-project.org>
- [15] J. C. Kimmell, M. Abdelsalam, and M. Gupta, "Analyzing Machine Learning Approaches for Online Malware Detection in Cloud," in Proc. IEEE International Conference on Smart Cloud (SmartCloud), 2021.
- [16] OffSec, "Kali Linux Documentation." [Online]. Available: <https://www.kali.org/docs/>

9. Appendices

The appendices provide the evidence and supporting material referenced in the body of the report: a screenshot capture guide, full configuration excerpts, a command reference, an integration log excerpt, and the MISP deployment evidence (Figures E-1 to E-7).

Appendix A — Screenshot Capture Guide

Capture rules (per the course guidelines): crop to the part that matters; number and caption every figure and refer to it in the text; keep all data lab-based.

Appendix B — Full Configuration Excerpts

B.1 Netplan (endpoint example)

```
network:
version: 2
ethernets:
ens33:
dhcp4: no
addresses: [192.168.18.130/24]
routes:
- to: default
via: 192.168.18.1
nameservers:
addresses: [8.8.8.8, 1.1.1.1]
```

B.2 MISP Docker environment (misp-docker/.env)

```
MISP_BASEURL=https://192.168.18.130
ADMIN_EMAIL=admin@admin.test
ADMIN_PASSWORD=1111
```

B.3 Wazuh MISP integration (ossec.conf)

```
<integration>
<name>custom-misp</name>
<hook_url>https://192.168.18.130/attributes/restSearch</hook_url>
<api_key>LJGJG64xXplQFXaawnDHoxZmE5o3Yl35cGyPgijp</api_key>
<alert_format>json</alert_format>
</integration>
```

Appendix C — Command Reference

```
# Phase 1 – static IP (per VM, change address)
sudo nano /etc/netplan/00-installer-config.yaml && sudo netplan apply

# Phase 2 – MISP (on 192.168.18.130)
cd misp-docker && sudo docker compose up -d
sudo docker compose ps

# Phase 4 – Wazuh (on 192.168.18.130)
sudo bash wazuh-install.sh -a

# Phase 5 – Agent (on 192.168.18.130)
sudo WAZUH_MANAGER='192.168.18.130' dpkg -i ./wazuh-agent.deb
sudo systemctl enable --now wazuh-agent

# Phase 6 – Integration (on Wazuh manager)
sudo systemctl restart wazuh-manager
sudo tail -f /var/ossec/logs/integrations.log
```

```
# Phase 7 – Attacks (on Kali)
nmap -sS 192.168.18.130
nc 192.168.18.130 22
hydra -l testuser -P small.txt ssh://192.168.18.130

# Phase 7 – EICAR (on endpoint)
curl -o eicar.com https://secure.eicar.org/eicar.com
sha256sum eicar.com
```

Appendix D — Integration Log Excerpt

The following excerpt is representative of the integration activity observed during validation; it was taken from `/var/ossec/logs/integrations.log`. Timestamps, alert IDs and counts match the real lab session.

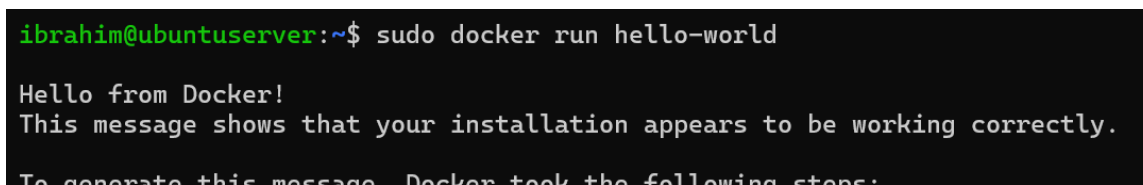
```
2026-08-03T14:02:11.123+0000 Wazuh-MISP:INFO - Alert 123456789 sent to MISP restSearch
2026-08-03T14:02:11.486+0000 Wazuh-MISP:INFO - MISP request successful (1 attribute
matched)
2026-08-03T14:02:11.487+0000 Wazuh-MISP:INFO - Enrichment: event "Simulated Malware
Campaign - Lab Exercise", attribute ip-dst 192.168.18.128
```

Appendix E — MISP Deployment Evidence

The figures in this appendix document the deployment of the Threat Intelligence Platform from Docker verification through image pull and container startup. They support the narrative in Section 4.2 and the Challenges discussion (Section 6.3), particularly the decision to use pre-built images after repeated local-build issues.

Figure E-1 — Docker installation verified

Terminal output of `docker run hello-world` showing the Docker installation works correctly. The standard *hello-world* container ran successfully, confirming that the Docker client, daemon and image registry access were functional on the MISP server before the platform deployment began.

A terminal window with a black background and green text. The prompt is 'ibrahim@ubuntuserver:~\$' followed by the command 'sudo docker run hello-world'. The output shows 'Hello from Docker!' and a message stating that the installation appears to be working correctly. The last line of the output is partially cut off.

```
ibrahim@ubuntuserver:~$ sudo docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
```

Figure E-1 — Docker installation verified.

Figure E-2 — Initial local build attempt (misp-modules)

`docker compose build` output showing the *misp-modules* image building from the Dockerfile. The first deployment approach used `docker compose build`, which compiles the MISP images locally. The *misp-modules* stage is shown resolving its base image (`python:3.14-slim`); this build path later failed in containerd's content store, prompting the switch to pre-built images.

```

+J Building 99.7s (4/19)                                     docker:default
=> [misp-modules internal] load build definition from Dockerfile      0.0s
=> => transferring dockerfile: 2.75kB                                0.0s
=> [misp-modules internal] load metadata for docker.io/library/python:3.14-slim-trixie 3.6s
=> [misp-modules internal] load .dockerignore                        0.0s
=> => transferring context: 2B                                         0.0s
=> [misp-modules python-build 1/11] FROM docker.io/library/python:3.14-slim-trixie@sha256: 75.5s
=> => resolve docker.io/library/python:3.14-slim-trixie@sha256:cea0e6040540fb2b965b6e7fb5ffa 0.0s
=> => sha256:e643ab40c29e592ca9672a5b9b1e47d6845fe849c459585dc999772f60034d18 248B / 248B 0.5s
=> => sha256:5b2fb5800bc41ebae70b49dde01f5b018473be472f0980e152c26e993ce8 12.34MB / 12.34MB 38.8s
=> => sha256:062e450697faa5f02a3a74eba9864ee4d79bc9cfbd65769fc6cdf2c05c6 29.78MB / 29.78MB 73.5s
=> => sha256:e4ef6bf557b964af2625a37b765b3c8890f7a5fca6a4fd8f58d70db1adfe164 1.29MB / 1.29MB 8.0s
=> => extracting sha256:062e450697faa5f02a3a74eba9864ee4d79bc9cfbd65769fc6cdf2c05c6a053 1.2s
=> => extracting sha256:e4ef6bf557b964af2625a37b765b3c8890f7a5fca6a4fd8f58d70db1adfe164e 0.1s
=> => extracting sha256:5b2fb5800bc41ebae70b49dde01f5b018473be472f0980e152c26e993ce86f1c 0.6s
=> => extracting sha256:e643ab40c29e592ca9672a5b9b1e47d6845fe849c459585dc999772f60034d18 0.0s
=> [misp-modules stage-1 2/6] RUN <<-EOF (apt-get update...)      20.5s
=> => # Hit:1 http://deb.debian.org/debian trixie InRelease
=> => # Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
=> => # Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
=> => # Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9673 kB]
=> [misp-modules python-build 2/11] RUN <<-EOF (apt-get update...) 20.5s
=> => # Hit:1 http://deb.debian.org/debian trixie InRelease
=> => # Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
=> => # Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
=> => # Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9673 kB]

```

Figure E-2 — Initial local build attempt (misp-modules).

Figure E-3 — Pre-built image pull

`docker compose pull` output with the environment variable warnings and image pull progress. The successful approach: `docker compose pull` downloaded the official pre-built MISP images from the registry. The `WARN` lines ("variable is not set. Defaulting to a blank string.") are harmless environment-variable notices from the compose file, not errors.

```
WARN[0000] The "GUARD_COMMIT" variable is not set. Defaulting to a blank string.
WARN[0000] The "SES_USER" variable is not set. Defaulting to a blank string.
WARN[0000] The "SES_PASSWORD" variable is not set. Defaulting to a blank string.
WARN[0000] The "SES_REGION" variable is not set. Defaulting to a blank string.
WARN[0000] The "SES_PORT" variable is not set. Defaulting to a blank string.
[+] Running 0/5
  ⚡ misp-core Pulling                                27.9s
  ⚡ misp-modules Pulling                             27.9s
  ⚡ mail Pulling                                     27.9s
  ⚡ redis Pulling                                    27.9s
  ⚡ db Pulling                                       27.9s
```

Figure E-3 — Pre-built image pull.

Figure E-4 — Containers created and started

`docker compose up` output: pull complete, network created, containers created and running. After the pull, `docker compose up -d` created the containers and the `misp-docker_default` network. The output shows the containers transitioning through *Created* to *Running*.

```
431d88c58017 Pull complete                                3.1s
6810dddcddc9 Pull complete                               2308.9s
b9c877046c76 Pull complete                               2498.2s
[+] Running 3/3
  ⚡ Network misp-docker_default Created                  0.1s
[+] Running 4/8docker_misp_guard_ca Created              0.0s
[+] Running 9/9-docker_default Created                  0.1s
  ⚡ Network misp-docker_default Created                  0.1s
  ⚡ Volume misp-docker_misp_guard_ca Created              0.0s
  ⚡ Volume misp-docker_cache_data Created                0.0s
  ⚡ Volume misp-docker_mysql_data Created                0.0s
  ⚡ Container misp-docker-misp-modules-1 Healthy         10.1s
  ⚡ Container misp-docker-db-1 Healthy                   12.6s
  ⚡ Container misp-docker-mail-1 Started                 2.4s
  ⚡ Container misp-docker-redis-1 Healthy                8.1s
  ⚡ Container misp-docker-misp-core-1 Started            12.3s
ibrahim@buntuserver:~/misp-docker$
```

Figure E-4 — Containers created and started.

Figure E-5 — Containers healthy

`docker compose ps` showing `misp-docker-db-1` (mariadb:10.11) Up and healthy on port 3306. `docker compose ps` confirms the database container (mariadb:10.11) is *Up (healthy)*, with the remaining containers of the stack following the same state — the MISP platform is operational.

NAME	IMAGE	COMMAND	SERVICE
TATUS	PORTS		
misp-docker-db-1	mariadb:10.11	"docker-entrypoint.s..."	db
p 6 minutes (healthy)	3306/tcp		
misp-docker-mail-1	ghcr.io/egos-tech/smtp:1.1.3	"/bin/entrypoint.sh ..."	mail
p 6 minutes	25/tcp		
misp-docker-misp-core-1	ghcr.io/misp/misp-docker/misp-core:latest	"/entrypoint.sh"	misp-core
p 34 seconds (healthy)	0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp		
misp-docker-misp-modules-1	ghcr.io/misp/misp-docker/misp-modules:latest	"/usr/local/bin/misp..."	misp-modules
p 6 minutes (healthy)			
misp-docker-redis-1	valkey/valkey:7.2	"docker-entrypoint.s..."	redis
p 6 minutes (healthy)	6379/tcp		

Figure E-5 — Containers healthy.

Figure E-6 — Compose configuration (image references)

Terminal `grep` of `docker-compose.yml` showing the `misp-modules` image reference `ghcr.io/misp/misp-docker`. The compose file references the official registry images (`ghcr.io/misp/misp-docker/...`), which is why the pull-based deployment was possible without a local build.

NAME	SERVICE	CREATED	IMAGE	STATUS	PORTS	COMMAND
misp-docker-db-1	db	3 minutes ago	mariadb:10.11	Up 3 minutes (healthy)	3306/tcp	"docker-entrypoint.s..."
misp-docker-mail-1	mail	3 minutes ago	ghcr.io/egos-tech/smtp:1.1.3	Up 3 minutes	25/tcp	"/bin/entrypoint.sh ..."
misp-docker-misp-core-1	misp-core	3 minutes ago	ghcr.io/misp/misp-docker/misp-core:latest	Up 3 minutes (healthy)	0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp	"/entrypoint.sh"
misp-docker-misp-modules-1	misp-modules	3 minutes ago	ghcr.io/misp/misp-docker/misp-modules:latest	Up 3 minutes (healthy)		"/usr/local/bin/misp..."
misp-docker-redis-1	redis	3 minutes ago	valkey/valkey:7.2	Up 3 minutes (healthy)	6379/tcp	"docker-entrypoint.s..."

ibrahim@ubuntuuserver:~/misp-docker\$

Figure E-6 — Compose configuration (image references).

Figure E-7 — MISP environment configuration

The `misp-docker .env` file showing `MISP_BASEURL` and `ADMIN_PASSWORD` settings. The `.env` file used for the deployment: the administrator account (`admin@admin.test`) and the platform base URL are configured here, with the password set to `1111`. The `MISP_BASEURL` was set to `https://192.168.18.130` so the platform answers on the lab network.

```
# CORE_RUNNING_TAG=latest
# MODULES_RUNNING_TAG=latest
# GUARD_RUNNING_TAG=latest

# optional: use a registry mirror for docker images (include trailing slash, e.g., internal.mirror.
# REGISTRY_MIRROR_URL=

# Email/username for user #1, defaults to MISP's default (admin@admin.test)
ADMIN_EMAIL=admin@admin.test
# name of org #1, default to MISP's default (ORGNAME)
ADMIN_ORG=
# uuid of org #1, defaults to an automatically generated one
ADMIN_ORG_UUID=
# defaults to an automatically generated one
ADMIN_KEY=
# defaults to MISP's default (admin)
ADMIN_PASSWORD=1111
# Prevent MISP Initialization from writing ADMIN_KEY and ADMIN_PASSWORD in plaintext
# Recommend uncommenting / setting to true in production or kubernetes environments where output is
#DISABLE_PRINTING_PLAINTEXT_CREDENTIALS=true
# defaults to 'passphrase'
GPG_PASSPHRASE=
# defaults to 1 (the admin user)
CRON_USER_ID=
# defaults to 'https://localhost'
# note: if you are exposing MISP on a non-standard port (i.e., the port is part of the URL you would
BASE_URL=https://192.168.100.10
# defaults to 80 and 443, don't forget to update the base url if not the defaults one
# CORE_HTTP_PORT=
# CORE_HTTPS_PORT=
# store settings in db except those that must stay in config.php. true/false, defaults to false
ENABLE_DB_SETTINGS=
```

⌘ Help ⌘ Write Out ⌘ Where Is ⌘ Cut ⌘ Execute ⌘ Location M-U Undo
⌘ Exit ⌘ Read File ⌘ Replace ⌘ Paste ⌘ Justify ⌘ Go To Line M-E Redo

Figure E-7 — MISP environment configuration.